Zadanie nr: **PW-1**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Transport cegieł.

Założenia:

Przy taśmie transportowej pracuje trzech pracowników oznaczonych przez P1, P2 i P3. Pracownicy wrzucają na początek taśmy cegły o masach odpowiednio 1, 2 i 3 jednostki masy. Na końcu taśmy stoi ciężarówka o ładowności ***C*** sztuk, którą należy zawsze załadować do pełna. Wszyscy pracownicy starają się układać cegły na taśmie najszybciej jak to możliwe. Taśma może przetransportować w danej chwili maksymalnie ***K*** sztuk cegieł. Jednocześnie jednak taśma ma ograniczony udźwig: maksymalnie ***M*** jednostek masy, tak, że niedopuszczalne jest położenie np. samych tylko cegieł najcięższych (3K>M). Po zapełnieniu ciężarówki na jej miejsce pojawia się nowa o takich samych parametrach. Na czas zmiany samochodu wstrzymywana jest praca taśmy.

Zadanie nr: **PW-2**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Symulacja obsługi kas w supermarkecie.

Założenia:

- M – liczba kas;
- N – maksymalna liczba klientów w sklepie (generowanie klientów);
- Dwa algorytmy obsługi klientów:
  - Osobne kolejki do każdej kasy;
  - Jedna wspólna kolejka;
- Przerwy kasjerów oraz ich zmiana po upływie określonego czasu (kasa jest w tym czasie zamknięta);
- Przed zamknięciem kasy należy dokończyć obsługę klienta przy kasie oraz wszystkich klientów z kolejki (jeżeli kolejki osobne).
- Symulacja zakupów przez klientów zanim przejdą do kasy.

Zadanie nr: **PW-3**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Symulacja obsługi na stacji paliw.

Założenia:

- Różne rodzaje paliw.
- Ubywanie paliwa ze stanu i po przekroczeniu rezerwy zaplanowanie uzupełnienia po ustalonym czasie.
- Zamykanie stacji na pewien czas w celu uzupełnienia zapasów.
- M – liczba stanowisk z dystrybutorami (na stanowisku może być kilka różnych dystrybutorów).
- K – liczba kas.
- N – maksymalna liczba samochodów, które mogą przebywać na stacji.

Zadanie nr: **PW-4**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Symulacja ruchu na stoku narciarskim.

Założenia:

- 3 trasy zjazdowe obsługiwane przez 3 wyciągi:
  - wyciąg ze stacji bazowej na szczyt;
  - wyciąg ze stacji bazowej do stacji w połowie stoku;
  - wyciąg ze stacji w połowie stoku na szczyt;
- Każdy wyciąg ma określoną pojemność i częstotliwość kursowania;
- Po upływie pewnego czasu dany wyciąg jest zamykany w celu dokonania prac serwisowych (wyciąg musi być pusty).

Zadanie nr: **PW-5**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Zakład produkcyjny.
Założenia:
Zakład produkcyjny produkuje na kilku liniach wyroby tego samego typu korzystając z dwóch rodzajów surowców.
Surowce różnego rodzaju przechowywane są w oddzielnych magazynach, które zaopatrywane są przez jeden samochód w sposób wyłączny.
Surowce danego rodzaju dowożone są po spadku zapasu w magazynie poniżej ustalonego poziomu.
Produkcja wyrobów na linii zajmuje losowy czas. Wyroby gotowe trafiają z linii do magazynu wyjściowego, z którego co losowy czas są zabierane w formie paczek (ustalona liczba sztuk).

Zadanie nr: **PW-6**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Zakład fryzjerski.
Założenia:
Zakład fryzjerski realizuje N rodzajów usług (np.: strzyżenie, modelowanie, golenie, …) i obsługuje klientów przybywających do niego w losowych odstępach czasu.
Klienci trafiają najpierw do poczekalni o ustalonej pojemności i czekają na wykonanie usługi wybranego rodzaju.
W zakładzie pracuje P fryzjerów, którzy są wyspecjalizowani w określonych usługach:

a. - P_1 – liczba fryzjerów wyspecjalizowanych w strzyżeniu,
b. - P_2 – liczba fryzjerów wyspecjalizowanych w modelowaniu,
c. - P_3 – liczba fryzjerów wyspecjalizowanych w goleniu,

(P_1, P_2, P_3, … <= P).
Do dyspozycji fryzjerów jest L foteli (L < P).

Zadanie nr: **PW-7**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Promy na rzece.
Założenia:
Przeprawę obsługuje N promów o określonej pojemności każdy.
Każda przystań ma ograniczoną pojemność.
Na przystań i z przystani prowadzi jedna droga, po której poruszają się pojazdy z różną prędkością.
Na promy jest tylko jeden wspólny wjazd i zjazd, przez który samochody wjeżdżają/zjeżdżają pojedynczo (pierwszeństwo samochodów zjeżdżających).
Promy czekają na zapełnienie przez określony maksymalny czas, po którym:
- rozpoczynają przeprawę, gdy na pokładzie znajduje się co najmniej jeden pojazd,
- lub rozpoczynają kolejny okres oczekiwania, gdy prom jest pusty.
Jeżeli zapełnią się wcześniej, to rozpoczynają przeprawę przed upływem tego czasu.
W przypadku przybycia promu do przystani, gdy jest zajęta przez inny prom, to oczekuje w kolejce.
Co pewien czas prom podlega przeglądowi serwisowemu.

Zadanie nr: **PW-8**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

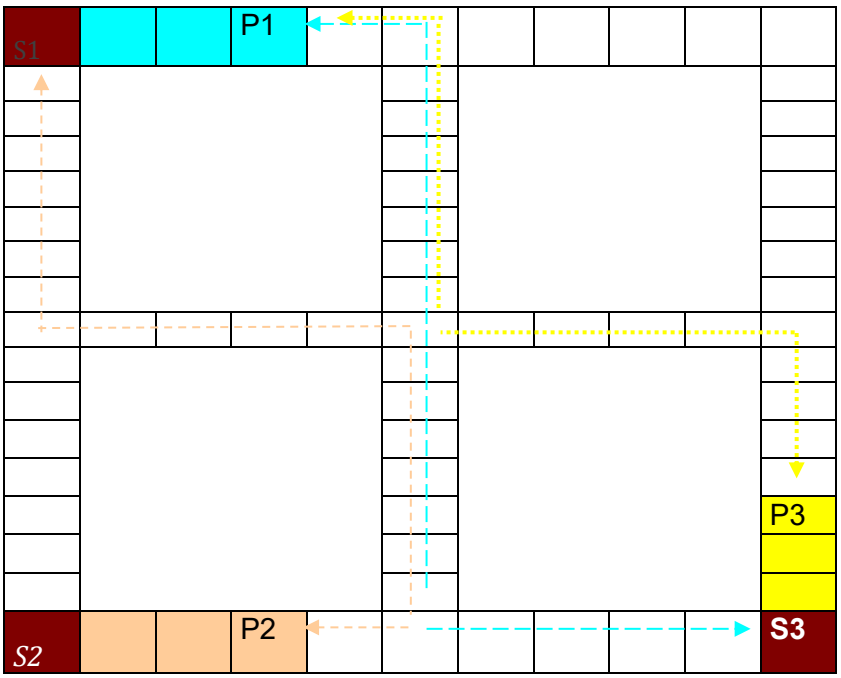
1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Metro.

Założenia:

- Po tunelach poruszają się 3 składy P1, P2, P3 o ustalonych długościach (K1, K2, K3).
  - długość składu w zakresie od 1 do wartości niepowodującej zablokowania.
- Należy zapewnić, że składy się nigdy nie zderzą zakładając, że każdy skład porusza się wyznaczoną trasą pomiędzy ustalonymi stacjami;
- Plansza ma zadane wymiary zgodnie z rysunkiem;
- Kierunek poruszania się składów jest ustalony.



Zadanie nr: **PW-9**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

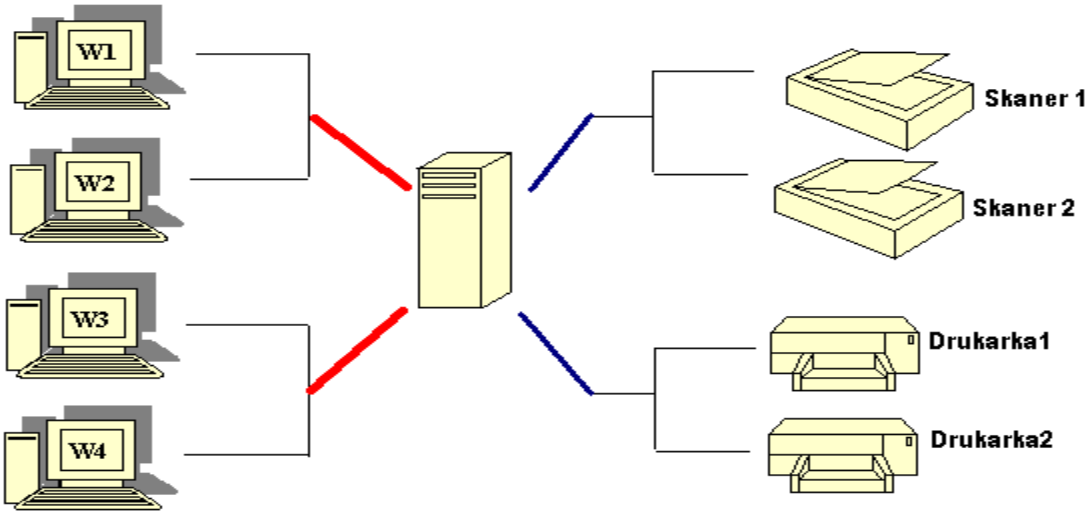
1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Sieć komputerowa.

Założenia:

W systemie zarejestrowane są 4 stacje robocze W1, W2, W3, W4 (zgrupowanie parami) każda ze stacji generuje żądanie korzystania z zasobów systemu. W danej chwili co najwyżej jeden komputer z danej grupy może korzystać z zasobów udostępnionych w danej grupie zasobów tzn. np. W1 korzysta ze Skanera 1 oraz W2 z Drukarki 2, nie dopuszcza się jednak takich sytuacji, w których W1 korzysta z Drukarki 1 oraz W2 korzysta z Drukarki 2 (ta sama grupa zasobów).



Zadanie nr: **PW-10**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Zakład usługowy.

Założenia:

Pewien zakład usługowy zatrudnia łącznie 4 pracowników. Jedna osoba przyjmuje zamówienia na naprawę sprzętu X a trzy pozostałe osoby realizują te naprawy. Zakład może maksymalnie pomieścić 100 elementów sprzętu X. Osoba przyjmująca zamówienia notuje adresy właścicieli oddawanego przez nich sprzętu do naprawy i dołącza je do przyjmowanego egzemplarza. Potem odkłada sprzęt na półkę. Gdy któryś z trzech "fizycznych" naprawiaczy jest wolny to bierze odebrany element, naprawia go, opakowuje, przykleja adres odbiorcy i wysyła do zamawiającego naprawę. Dopiero wówczas element znika z ewidencji zakładu naprawczego.

Zadanie nr: **PW-11**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Statek.

Założenia:

Przy nabrzeżu stoi statek o pojemności N. Statek z lądem jest połączony mostkiem o pojemności K (K<N). Na statek próbują dostać się pasażerowie, z tym, że na statek nie może ich wejść więcej niż N, a wchodząc na statek na mostku nie może być ich równocześnie więcej niż K. Statek co jedną godzinę wypływa w rejs. W momencie odpływania kapitan statku musi dopilnować, aby na mostku nie było żadnego wchodzącego pasażera. Jednocześnie musi dopilnować by liczba pasażerów na statku nie przekroczyła N. Napisać odpowiednio procedury Pasażer i Kapitan zsynchronizowane za pomocą jakiejkolwiek z metod synchronizacji procesów.

Zadanie nr: **PW-12**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

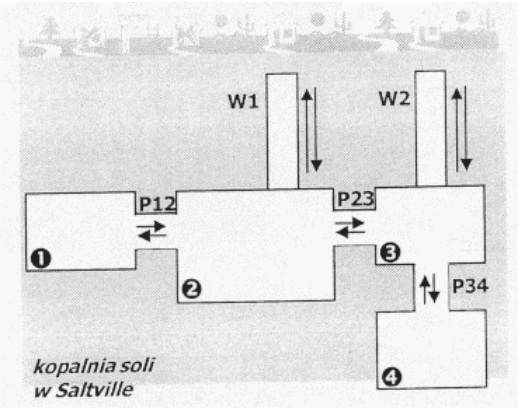
1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Kopalnia.

Założenia:

W mieście Saltville postanowiono udostępnić turystom starą kopalnię soli. W kopalni tej znajdują się 4 wspaniałe sale warte obejrzenia. Sale połączone są przejściami zgodnie z planem podanym poniżej. Każda z sal ma pojemność X osób, z wyjątkiem sali nr 2, która jest 2 razy większa od pozostałych. Kopalnię można zwiedzać tylko wg ustalonego planu A lub B. Plan A zakłada zwiedzanie sal w kolejności 1,2,3,4, wjazd do kopali windą W1, wyjazd W2. Plan B dokładnie odwrotnie czyli zwiedzanie sal w kolejności 4,3,2,1, wjazd do kopali windą W2, a wyjazd W1.



Napić procedury dla turysty zwiedzającego kopalnię wg trasy A lub B. Uwagi:

- Nie wolno zatrzymywać się w przejściu pomiędzy salami;
- Przejścia są dwukierunkowe;
- Przejścia są 1-osobowe;
- Przez salę można przechodzić nie zwiedzając jej;
- Nie wolno dzielić sali na strefy o ograniczonym dostępie.

Zadanie nr: **PW-13**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Parowozownia.

Założenia:

W pewnej parowozowni istnieje M miejsc remontowych (równocześnie może być naprawianych M lokomotyw). Naprawa taka trwa pewien nieznany, ale skończony okres czasu, po czym lokomotywa opuszcza parowozownię i rozpoczyna się jej normalna eksploatacja. Cykl ten powtarza się. Aby wjechać do parowozowni, należy przejechać przez obrotową nastawnię. Do nastawni prowadzą jedynie dwa tory, którymi można wjechać na teren parowozowni. Należy zagwarantować, że nie dojdzie do sytuacji, w której na jednym odcinku toru znajdą się 2 lokomotywy, gdyż grozi to zderzeniem. Należy założyć że pracownik obsługujący nastawnię posiada wiedzę, która pozwala mu operować nastawnią w ten sposób, iż wybierane będą, o ile są dostępne, wolne tory prowadzące do parowozowni lub do bram. Co pewien czas w parowozowni musi zostać przeprowadzony remont. Decyzję o tym podejmuje szef robót remontowych. On także decyduje o jego zakończeniu. Dla bezpieczeństwa robotników, w czasie remontu żadna lokomotywa nie może znajdować się na terenie parowozowni.

Zadanie nr: **PW-14**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

UI.

Założenia:

Rój pszczół liczy początkowo N osobników. W danym chwili w ulu może znajdować się co najwyżej K (K<N/2) pszczół. Pszczoła, która chce dostać się do wnętrza ula, musi przejść przez jedno z dwóch istniejących wejść. Wejścia te są bardzo wąskie, więc możliwy jest w nich jedynie ruch w jedną stronę w danej chwili czasu. Zbyt długotrwałe przebywanie w ulu grozi przegrzaniem, dlatego każda z pszczół instynktownie opuszcza ul po pewnym skończonym czasie. Znajdująca się w ulu królowa, co pewien czas składa jaja, z których wylęgają się nowe pszczoły. Aby jaja zostały złożone przez królową, w ulu musi istnieć wystarczająca ilość miejsca. (1 jajo liczone jest tak jak 1 dorosły osobnik). Napisz program pszczoły "robotnicy" i królowej, tak by zasymulować cykl życia roju pszczół. Każda z pszczół "robotnic" umiera po pewnym określonym czasie X, liczonym ilością odwiedzin w ulu.

Zadanie nr: **PW-15**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Basen.

Założenia:

W pewnym miasteczku znajduje się kompleks basenów krytych. W jego skład wchodzą: basen olimpijski, basen rekreacyjny i brodzik dla dzieci. Na każdym z nich może przebywać w danej chwili maksymalnie X1, X2 i odpowiednio X3 osób płacących. Osoba odwiedzająca centrum kupuje bilet czasowy, który upoważnia ją do korzystania z dowolnego basenu zgodnie z regulaminem pływalni przez pewien określony czas. Dzieci poniżej 10. roku życia nie płacą za bilet. Istnieje pewna liczba osób (VIP) posiadająca karnety wstępu, które umożliwiają wejście na pływalnie z pominięciem kolejki oczekujących. Chętni do wejścia na basen, w różnym wieku (od 1 roku do 70 lat), pojawiają się w sposób losowy co pewien czas. Okresowo centrum jest zamykane, aby umożliwić obsłudze wymianę wody we wszystkich basenach równocześnie. W trakcie tej czynności nikt nie ma prawa przebywać na terenie pływalni (należy poczekać aż wszyscy opuszczą basen). Napisz programy kasjera i klientów, który zagwarantuje sprawną obsługę i korzystanie z obiektu zgodnie z niżej podanym regulaminem. Podpowiedź: Zaimplementuj dziecko i jego opiekuna jako jeden proces.

Regulamin pływalni:

- Tylko osoby pełnoletnie mogą korzystać z basenu olimpijskiego.
- Dzieci poniżej 10. roku życia nie mogą przebywać na basenie bez opieki osoby dorosłej.
- W brodziku kąpać mogą się jedynie dzieci do 5. roku życia i ich opiekunowie.
- Dzieci do 3. roku życia muszą pływać w Pampersach J.
- Średnia wieku w basenie rekreacyjnym nie może być wyższa niż 40 lat.
- Noszenie czepków pływackich nie jest obowiązkowe.

Zadanie nr: **PW-16**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Pizzeria.

Założenia:

W pizzerii znajduje się odpowiednio X1, X2, X3, X4 stolików 1,2,3 lub 4-osobowych. Klienci mogą usiąść przy stoliku pod warunkiem wcześniejszego zamówienia i otrzymania pizzy (czas od zamówienia do otrzymania pizzy jest krótki i może być traktowany jako nieistotny). Niedopuszczalne jest, by klient musiał czekać z gorącą pizzą na miejsce siedzące. Klienci przychodzą do lokalu sami lub w grupach 2,3-osobowych. Z uwagi na liczne przypadki kłótni przy stołach o ketchup (darmowy), różne grupy klientów nie mogą siadać przy tym samym stoliku, chyba że są równoliczne. Napisać program klienta pizzerii (klientem jest grupa znajomych) i szefa lokalu (kasjera), który jest odpowiedzialny za prawidłowe i możliwe najbardziej dochodowe funkcjonowanie lokalu.

Zadanie nr: **PW-17**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Supermarket.

Założenia:

W pewnym supermarkecie jest łącznie M kas (M > 3). Zasady ich działania przyjęte przez kierownika sklepu są następujące:

- Zawsze działają min. 2 stanowiska kasowe.
- Na każdych K klientów znajdujących się na terenie supermarketu powinno przypadać min. 1 czynne stanowisko kasowe.
- Jeśli liczba klientów jest mniejsza niż K*(N-1), gdzie N oznacza liczbę czynnych kas, to jedna z kas zostaje zamknięta.

Jeśli w kolejce do kasy czekali klienci (przed ogłoszeniem decyzji o jej zamknięciu) to powinni zostać obsłużeni przez tę kasę. Klienci przychodzą do supermarketu w losowych momentach czasu i przebywają w nim przez pewien określony losowy dla każdego z nich czas. Napisz program klienta i kierownika kasjerów.

Zadanie nr: **PW-18**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Stadion.

Założenia.

Na stadionie piłkarskim rozegrany ma zostać mecz finałowy Ligi Mistrzów. Z uwagi na rangę imprezy ustalono następujące rygorystyczne zasady bezpieczeństwa.

- Na stadionie może przebywać maksymalnie K kibiców.
- Wejście na stadion możliwe będzie tylko po przejściu drobiazgowej kontroli, mającej zapobiec wnoszeniu przedmiotów niebezpiecznych.
- Kontrola przy wejściu jest przeprowadzana równolegle na 3 stanowiskach, na każdym z nich mogą znajdować się równocześnie maksymalnie 3 osoby.
- Jeśli kontrolowana jest więcej niż 1 osoba równocześnie na stanowisku, to należy zagwarantować, by byli to kibice tej samej drużyny.
- Kibic oczekujący na kontrolę może przepuścić w kolejce maksymalnie 5 innych kibiców. Dłuższe czekanie wywołuje jego frustrację i agresywne zachowanie, którego należy unikać za wszelką cenę.

Zadanie nr: **PW-19**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Statki wpływające do portu.

Założenia:

Do portu zawijają i odpływają z niego statki. Każdy z n statków po wpłynięciu do portu musi zająć jakieś wolne miejsce przy nabrzeżu. Liczba miejsc jest ograniczona i wynosi m ($m < n$). Ponadto, w celu wejścia do portu lub wyjścia z portu statek i-ty potrzebuje k_i holowników. Łączna liczba holowników wynosi h ($h < \sum_{i=1}^{n} k_i$). Napisać program dla procesu-statku zapewniający, że nie nastąpi zakleszczenie ani zagłodzenie któregoś ze statków. Miejsca przy nabrzeżu i holowniki należy traktować jako zasoby.

Zadanie nr: **PW-20**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Kamieniołom.

Założenia:

Pracownicy kamieniołomu układają 3 typy bloków kamiennych na palecie (każdy swój typ). Bloki mają ustandaryzowane rozmiary o jednostkowej szerokości i wysokości. Długości bloków wynoszą 1, 2 lub 3 jednostki. Paleta ma rozmiar 3x3 jednostki (zob. rys.1). Bloki układane są warstwami. Każda warstwa musi być zapełniona całkowicie i zabezpieczona izolatorem. Pracownik sterujący ładowaniem układa izolator po ułożeniu każdej warstwy. Po ułożeniu 3 warstw gotowa paleta jest wysyłana transporterem.

```
  |  |
--+--+--
  |  |
--+--+--
  |  |
```

Rys. 1

Dodatkowym ograniczeniem przy układaniu bloków jest ich masa. Bloki o długościach 1, 2 i 3 mają odpowiednio masę m1=1, m2=3 i m3=5 jednostek. Najniższa warstwa na palecie może mieć maksymalną sumaryczną masę M1=14 jednostek, warstwa druga M2=13 jednostek, a najwyższa warstwa M3=11 jednostek.

Zadanie nr: **PW-21**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Symulacja układu wind w biurowcu.

Założenia:

- Mamy 3 rodzaje wind i N pięter;
  - 1. rodzaj obsługuje piętra od 0 do [N/2];
  - 2. rodzaj obsługuje piętra od [N/2]+1 do N oraz 0;
  - 3. rodzaj obsługuje poziomy ujemne (parkingi) oraz piętro 0;
- Pasażer losuje numer piętra i próbuje się na nie dostać;
- Winda może pomieścić M osób.

Zadanie nr: **PW-22**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Załadunek koparkami.

Założenia:

Napisać program synchronizację podstawiania samochodów pod załadunek koparką. Należy założyć istnienie N koparek i K samochodów o małej ładowności i L samochodów o dużej ładowalności. Jedna koparka obsługuje równocześnie dwa małe pojazdy lub jeden duży. Dodatkowo należy przyjąć, że operatorzy koparek mają przerwę obiadową między godziną 13^{00} i 14^{00}, a niecierpliwi kierowcy rezygnują z załadunku jeśli nie podjadą na stanowisko w ciągu założonego maksymalnego czasu oczekiwania np. 0.5 godz.

Zadanie nr: **PW-23**

Język implementacji: **Java**
Środowisko implementacyjne: **Eclipse, Intelij IDEA, Netbeans**
Termin wykonania: **ostatnie zajęcia**

Podstawowe wymagania:

a. liczba wątków powinna być dobrana z wyczuciem tak, aby zachować czytelność interfejsu i jednocześnie umożliwić zobrazowanie reprezentatywnych przykładów,
b. kod źródłowy programu musi być tak skonstruowany, aby można było „swobodnie" modyfikować liczbę wątków (za wyjątkiem zadań o ściśle określonej liczbie procesów),
c. graficzne zobrazowanie działania procesów współbieżnych,
d. odczyt domyślnych danych wejściowych ze sformatowanego, tekstowego pliku danych (xml, properties, inne),
e. możliwość modyfikacji danych wejściowych poprzez GUI.

Sprawozdanie (w formie elektronicznej) powinno zawierać następujące elementy:

1) stronę tytułową,
2) niniejszą treść zadania,
3) syntetyczny opis problemu – w tym wszystkie przyjęte założenia,
4) wykaz własnych procesów sekwencyjnych,
5) wykaz współdzielonych zasobów,
6) wykaz wyróżnionych punktów synchronizacji,
7) wykaz obiektów synchronizacji,
8) listing programu.

Problem do rozwiązania:

Most.

Założenia:

Na drodze dwukierunkowej znajduje się wąski most, przez który w danej chwili mogą jechać samochody tylko w jednym kierunku. Napisać program współbieżny synchronizujący przejazd samochodów jadących z przeciwnych kierunków, tak aby nie było kolizji i żeby samochód z każdego kierunku mógł w końcu przejechać przez most. Czasy pojawiania się samochodów i przejazdu przez most generować z rozkładów losowych o parametrach wprowadzanych przez użytkownika. Należy zaproponować co najmniej dwa rozwiązania.